TASK 9 — NUMPY ARRAY CREATION AND DATA TYPES

Expanded • Clear • Explained • Submission-Ready

1. Task Overview

The task focuses on creating NumPy arrays and exploring their **shape, dimensions, size, data types, and basic numerical operations**. It introduces NumPy as a core Python library used in data science and numerical data processing.

Source task requirements: one-dimensional and two-dimensional arrays, demonstration of array attributes, numerical operations, comparison of NumPy arrays with Python lists, suggested Iris and Student Marks datasets, and basic NumPy interview questions.

2. Objective

Learn the fundamentals of NumPy, a core Python library for data science, by creating arrays, examining their properties, and applying basic numerical operations.

3. Tools Used

• Python
• NumPy
• Jupyter Notebook

4. Deliverables

• Examples of one-dimensional and two-dimensional arrays
• Demonstration of array attributes
• Examples of numerical operations
• Student marks example
• Iris-style numerical example
• NumPy and Python-list comparison
• Interview questions and answers

5. NumPy Basics

NumPy provides the **ndarray** structure for working with numerical data. Arrays can be one-dimensional, two-dimensional, or higher-dimensional. The task concentrates on the first two forms.

6. Creating a One-Dimensional Array

```
import numpy as np

one_d = np.array([10, 20, 30, 40, 50])
print(one_d)
```

Explanation: np.array() converts the supplied sequence into a NumPy array. This example creates a single row of five numerical values. A 1D array is useful for representing a single sequence such as marks, prices, temperatures, or measurements.

7. Creating a Two-Dimensional Array

```
two_d = np.array([
    [10, 20, 30],
    [40, 50, 60]
])

print(two_d)
```

Explanation: This array contains two rows and three columns. Two-dimensional arrays are useful for representing table-like numerical data.

8. Understanding Array Attributes

Attribute	Meaning	Example
shape	Size along each dimension	(2, 3)
ndim	Number of dimensions	2
size	Total number of elements	6
dtype	Data type of elements	int64 / float64

```
print(two_d.shape)
print(two_d.ndim)
print(two_d.size)
print(two_d.dtype)
```

Key point: These attributes allow a data analyst to quickly understand the structure and storage characteristics of an array.

9. Working with Data Types

```
integer_array = np.array([1, 2, 3], dtype=np.int32)
float_array = np.array([1, 2, 3], dtype=np.float64)

print(integer_array.dtype)
print(float_array.dtype)
```

Explanation: dtype specifies the type used for the array elements. The example explicitly creates an integer array and a floating-point array.

10. Basic Numerical Operations

```
marks = np.array([72, 85, 91, 64, 78])

print("Marks + 5:", marks + 5)
print("Marks * 2:", marks * 2)
print("Average:", marks.mean())
print("Maximum:", marks.max())
print("Minimum:", marks.min())
```

Explanation: NumPy supports array-based arithmetic. The same operation can be applied to multiple values without writing a separate loop for each element. mean(), max(), and min() provide basic statistical summaries.

11. Indexing and Slicing

```
print(marks[0])
print(marks[:3])
```

Indexing accesses one element using its position. **Slicing** selects a range of elements. Python indexing starts at zero, so marks[0] refers to the first value.

12. Useful Array-Creation Functions

```
zeros = np.zeros((2, 3))
ones = np.ones((2, 3))
sequence = np.arange(0, 10, 2)

print(zeros)
print(ones)
print(sequence)
```

zeros() creates an array filled with zero values. **ones()** creates an array filled with one values. **arange()** creates a numerical sequence using a start, stop, and step pattern.

13. Student Marks Example

```
marks = np.array([72, 85, 91, 64, 78])

average = marks.mean()
highest = marks.max()
lowest = marks.min()

print("Average:", average)
print("Highest:", highest)
print("Lowest:", lowest)
```

Purpose: This example demonstrates how NumPy can support basic student-performance analysis using numerical data.

14. Iris-Style Dataset Example

```
iris = np.array([
    [5.1, 3.5, 1.4, 0.2],
    [6.2, 3.4, 5.4, 2.3],
    [5.9, 3.0, 4.2, 1.5]
])

print("Shape:", iris.shape)
print("Column means:", iris.mean(axis=0))
```

Explanation: Each row represents one sample and each column represents a numerical feature. axis=0 calculates the mean down each column.

15. NumPy Array vs Python List

Feature	Python List	NumPy Array
Primary purpose	General-purpose collection	Numerical/data-science computing
Numerical operations	Often require loops or comprehensions	Supports array-based operations
Shape / dimensions	Not native array attributes	shape and ndim available
Data type	Can contain mixed Python objects	Generally homogeneous
Data science use	Useful for general collections	Designed for efficient numerical work

Important: The main advantage demonstrated in this task is convenient numerical processing with array-oriented operations.

16. Interview Questions and Clear Answers

Q1. What is NumPy?
NumPy is a Python library used for numerical computing and array-based data processing.

Q2. What is the difference between a NumPy array and a Python list?
NumPy arrays are designed for efficient numerical operations and generally store homogeneous data, while Python lists are general-purpose collections.

Q3. What does the shape attribute represent?
shape describes the size of the array along each dimension. For example, (2, 3) means two rows and three columns.

Q4. What does ndim represent?
ndim gives the number of dimensions of an array.

Q5. What does size represent?
size gives the total number of elements in the array.

17. File-by-File Explanation

File	What it contains / why it is used
Task_9.py	Main Python implementation. Contains array creation, attributes, data types, numerical operations, student marks, a
Task_9.ipynb	Interactive Jupyter Notebook version. Code can be executed cell-by-cell and outputs can be viewed directly.
README.md	Short project documentation covering description, objective, tools, deliverables, topics, skills, and status.
student_marks.csv	Small student marks dataset for numerical analysis practice.
iris_sample.csv	Small Iris-style dataset containing four numerical features per record.
Assessment PDF	Detailed submission document containing explanations, code examples, comparison, interview answers, skills, feed

18. How to Run the Project

Step 1: Install Python.
Step 2: Install NumPy using **pip install numpy** if it is not already installed.
Step 3: Run Task_9.py, or open Task_9.ipynb in Jupyter Notebook.
Step 4: Execute the code cells in order and review the outputs.
Step 5: Use the CSV files for additional practice.

19. Top 5 Skills Used

1. **NumPy**
2. **Python Programming**
3. **Numerical Data Processing**
4. **Array Manipulation**
5. **Data Analysis Fundamentals**